MODULE 15 · DOMAIN 3 - APPLICATION DEVELOPMENT

Frameworks: LangChain, LangGraph and the rest

What each framework is for, how they plug into Databricks, and why the choice matters less than the wrapper.

SECTION 3 of 6 EXAM WEIGHT 30% MODULE 15 of 40

admin@sthithapragna.com

A framework is a Lego set. Plain Python is a box of loose bricks. **Databricks does not care which you build with, as long as it fits the same base plate.**

admin@sthithapragna.com

What each framework is for

FRAMEWORK	BUILT FOR
LangChain	Composable components: prompts, models, retrievers, parsers, many integrations
LangGraph	Stateful graphs: loops, branches, memory, human approval steps
LlamaIndex	Data-centric rag: loaders, indexes, query engines
DSPy	Programs whose prompts are optimised automatically against a metric
OpenAI Agents SDK	Lightweight agents with tools and handoffs between agents
Plain Python	Full control, fewest dependencies, simplest cases

ON THE EXAM

Match the framework to the shape of the problem: a line, a loop, a data pipeline, or an optimisation.

admin@sthithapragna.com

LangChain or LangGraph?

LANGCHAIN CHAIN

A line

- Steps run once, in order
- Retrieve, prompt, generate, parse
- Simple to read and test
- The deterministic chain from module 01

LANGGRAPH GRAPH

A map with loops

- Steps can repeat or branch
- State carried between steps
- Checkpoints and human approval
- The agent pattern from modules 01 and 04

THE TIEBREAKER

If the flow needs a loop, a branch on results, state across steps, or a pause for approval, choose LangGraph.

admin@sthithapragna.com

A framework, or plain Python?

A FRAMEWORK WHEN

Standard patterns, many integrations

- Common rag or agent shapes
- Many connectors needed
- Team knows the framework
- Speed of development matters

PLAIN PYTHON WHEN

Simple, or very specific

- One model call and a lookup
- Few dependencies wanted
- Unusual control flow
- Every line must be auditable

THE TIEBREAKER

A framework is not required to build an agent on Databricks. The requirement is the ResponsesAgent wrapper, not the framework inside it.

admin@sthithapragna.com

When DSPy or LlamaIndex is the right answer

- **DSPy**: you have labelled examples and a metric, and want prompts and few-shot examples **optimised automatically**.
- **LlamaIndex**: the hard part is **ingesting and indexing** many document sources for rag.
- **OpenAI Agents SDK**: lightweight agents that **hand off** to one another.
- None of these is a model. Each still calls a model endpoint, such as a Databricks one.

> ON THE EXAM
>
> Optimise prompts against a metric: DSPy. Data ingestion and indexing: LlamaIndex. Branching, stateful agent: LangGraph.

The integration pieces

CLASS OR PACKAGE	WHAT IT CONNECTS
`ChatDatabricks`	LangChain chat model over a Databricks serving endpoint
`DatabricksEmbeddings`	LangChain embeddings over a Databricks embedding endpoint
`DatabricksVectorSearch`	LangChain retriever over an AI Search index
`UCFunctionToolkit`	Unity Catalog functions as LangChain tools
`DatabricksMCPServer`	Databricks MCP servers as tools
`DatabricksOpenAI`	OpenAI-style client for Databricks models

ON THE EXAM

The first five live in the databricks-langchain package. The last is in databricks-openai. Completion models are legacy: use chat models.

admin@sthithapragna.com

Any framework, one wrapper

01
Build
LangGraph, LangChain, OpenAI SDK or plain Python.

02
Wrap
As a ResponsesAgent.

03
Use
AI Playground, evaluation, tracing.

04
Deploy
Databricks Apps, or Model Serving.

ON THE EXAM

Databricks' own words: you can author agents using any framework. The key is wrapping the agent with ResponsesAgent.

Turning on MLflow tracing for a framework

CODE	TRACES
`mlflow.langchain.autolog()`	LangChain
`mlflow.langgraph.autolog()`	LangGraph
`mlflow.openai.autolog()`	OpenAI SDK and Agents SDK
`@mlflow.trace decorator`	Your own functions

ON THE EXAM

On serverless compute, autologging for these frameworks is not switched on for you. Call the autolog line explicitly.

Common mistakes in chain code

- Template variables that **do not match** the keys passed in: {question} versus query.
- Retrieved documents passed as objects, **not formatted** into context text.
- No output parser, so the chain returns a **message object**, not a string.
- A **legacy completion** class where a chat model is needed.
- Endpoint names or tokens **hard-coded**, instead of configuration and secrets.

> ON THE EXAM
>
> When chain code fails, check that names match first: template variables, input keys, endpoint names.

How Pennant Outdoor builds its returns agent

01
Framework
LangGraph: the returns flow loops and needs approval.

02
Model
ChatDatabricks over a served model.

03
Tools
UC functions via the MCP server.

04
Wrap
ResponsesAgent, for Databricks features.

05
Trace
mlflow.langgraph.autolog() called explicitly.

ON THE EXAM

The loop and the approval step chose LangGraph. Everything else is the same whichever framework is used.

> An agent must retrieve policy text, check whether it is sufficient, and re-query up to three times if not. Before issuing any refund, it must pause for a manager's approval.
>
> State, such as the claim details gathered so far, must carry across every step.

Which tool best fits?

A. A simple sequential LangChain chain, run from start to end

B. A LangGraph stateful graph with a loop and an approval step

C. A single, very detailed prompt with every step written out

D. A LlamaIndex index over the policy documents and claims

◆ PAUSE AND DECIDE

Which tool best fits?

A. Sequential LangChain chain
Runs once, in order. Cannot loop back or pause.

B. LangGraph graph
Loops, state across steps, and a pause for approval.

C. One detailed prompt
No real loop, no state, no genuine approval step.

D. LlamaIndex index
Handles the retrieval part only, not the control flow.

THE SIGNAL

Re-query up to three times, pause for approval, carry state. A loop, an interrupt and state are LangGraph's reason to exist.

> An engineer builds a LangGraph agent in a notebook on serverless compute and runs it several times.
>
> No traces appear in the MLflow experiment.

What is the most likely fix?

A Rewrite the agent in LangChain, which traces automatically

B Move the agent to a larger model

C Increase the Python logging level

D Call mlflow.langgraph.autolog() explicitly before running the agent

◆ PAUSE AND DECIDE

What is the most likely fix?

A Rewrite in LangChain

Same issue: autologging is off on serverless for all of them.

B Larger model

Model choice has nothing to do with tracing.

C Python logging level

MLflow traces are not Python log messages.

D Call the autolog line

On serverless, autologging must be switched on explicitly.

THE SIGNAL

Serverless compute, and no traces. Autologging for GenAI frameworks is not enabled automatically there.

> A LangChain prompt template reads: Answer {question} using only this context: {context}.
>
> The chain is invoked with a dictionary containing the keys query and context. It fails with an error about a missing variable.

What should the engineer fix?

A Make the input keys match the template variables

B Increase the model's maximum output tokens setting

C Rebuild the whole chain in LangGraph instead

D Add an output parser to the end of the chain

◆ PAUSE AND DECIDE

What should the engineer fix?

A Match keys to variables

The template wants question. The input supplies query.

B More output tokens

The chain fails before the model is ever called.

C Rebuild in LangGraph

Same mismatch would follow into any framework.

D Add an output parser

Parses the result. The error happens at the input.

THE SIGNAL

A missing-variable error means names do not match. Compare template variables with the keys passed in.

> A team has 2,000 labelled examples and an accuracy metric for a ticket-classification pipeline.
>
> Engineers spend hours hand-editing prompts and few-shot examples. The team wants these tuned automatically against the metric.

Which tool is designed for this?

A LangChain

B LlamaIndex

C DSPy

D OpenAI Agents SDK

◆ PAUSE AND DECIDE

Which tool is designed for this?

A **LangChain**
Composes components. Prompts are still hand-written.

B **LlamaIndex**
Focused on ingesting and indexing data.

C **DSPy**
Optimises prompts and examples against a metric.

D **OpenAI Agents SDK**
Builds agents with tools and handoffs. No prompt optimisation.

THE SIGNAL

Labelled examples, a metric, and prompts tuned automatically. That is DSPy's defining purpose.

What to carry forward

- **LangChain** for a line. **LangGraph** for loops, branches, state and approvals.
- **DSPy** to optimise prompts against a metric. **LlamaIndex** for data-heavy rag.
- Any framework works on Databricks if wrapped as a **ResponsesAgent**.
- databricks-langchain: ChatDatabricks, DatabricksEmbeddings, DatabricksVectorSearch, UCFunctionToolkit.
- On serverless, call the framework's autolog() line explicitly.

> ON THE EXAM
>
> Next: module 16 reads model responses critically, for quality and safety, before we build guardrails.